Retrieval-Augmented Code Generation (RACG)

Kristine Hambardzumyan **Anna Vardazaryan** **Margarita Sakoyan**
Yerevan State University, Department of Statistics
{qristine.hambardzumyan3, anna.vardazaryan3, margarita.sakoyan}@edu.ysu.com

Abstract

Large language models (LLMs) have recently shown promising results in automated program repair, generating code patches from natural-language bug descriptions. However, real-world software repositories pose a fundamental challenge: effective bug fixing requires identifying the correct modification location and reasoning over existing implementations within large codebases. SWE-Bench formalizes this challenge by evaluating code repairs on real repositories with execution-based test suites.

In this work, we study retrieval-augmented code generation for SWE-Bench Lite. We systematically evaluate how different repository context retrieval strategies affect patch correctness, while keeping the underlying code generation model fixed. Starting from a no-retrieval baseline, we evaluate lexical retrieval using BM25, hybrid retrieval that combines BM25 with embedding-based semantic reranking via reciprocal rank fusion, and dense semantic retrieval using FAISS-based similarity search. Our results show that retrieval does not uniformly improve resolution rates over the baseline on this sample, but it substantially affects which instances are solved. These findings clarify the role of retrieval in repository-scale program repair and motivate the use of stronger, execution-informed signals for context selection.

1 Introduction

Automated software repair has emerged as a promising application of large language models (LLMs). Given a natural-language description of a bug, modern code-oriented LLMs can often generate syntactically valid patches that resemble human-written fixes. Despite this progress, applying LLMs to real-world repositories remains challenging. Unlike standalone coding tasks, realistic bug fixing requires locating the relevant code in a large and unfamiliar codebase and making a minimal, semantically correct modification that integrates with existing logic.

SWE-Bench (Jiménez et al., 2024) captures this difficulty by framing bug fixing as a grounded task over real repositories. Each instance consists of a repository snapshot, a natural-language issue description, and an executable test suite used for evaluation. Solving such tasks is not merely a matter of code generation, but of context-aware reasoning: the model must identify where to intervene and how to adapt existing behavior without introducing regressions.

Although LLMs can generate plausible code patches from natural-language descriptions, effective repair in this setting is rarely limited by syntax alone. Instead, success depends on identifying the correct modification location within a large codebase and reasoning about how existing implementations should be minimally and correctly adapted. A fundamental limitation of LLMs is that they do not have direct access to repository contents unless relevant code is explicitly provided in the prompt. Because real-world repositories may contain thousands of files, it is infeasible to include the entire codebase within the model's context window.

Retrieval-augmented code generation (RACG) (Lewis et al., 2020) addresses this limitation by selectively retrieving relevant code snippets from the repository and injecting them into the model prompt, enabling the model to reason over the appropriate localized context during patch generation. However, the effectiveness of RACG depends critically on which code is retrieved and how it is presented. This leads to the central research question of this work: how should repository context be retrieved and selected so that an LLM is most likely to generate a correct and minimal patch?

In this work, we investigate the role of retrieval in LLM-based program repair on SWE-Bench Lite. We focus exclusively on the retrieval component, keeping the code generation model and agent in-

frastructure fixed. We begin with a no-retrieval baseline in which no repository code is injected into the model prompt. In this setting, the agent may still interact with the repository through shell commands provided by the `mini-SWE-agent` environment, but it receives no explicit retrieval-guided context. We then evaluate three retrieval strategies: (1) lexical retrieval using BM25, (2) hybrid retrieval that combines BM25 candidate selection with embedding-based semantic reranking via reciprocal rank fusion (RRF), and (3) dense semantic retrieval using hosted embedding models with FAISS similarity search and path-based weighting.

Our contributions are threefold:

- We provide a controlled empirical study of retrieval strategies for SWE-Bench Lite, isolating retrieval as the primary experimental variable.
- We implement and evaluate three retrieval approaches: BM25-based lexical retrieval, hybrid BM25–semantic fusion via reciprocal rank fusion, and dense semantic retrieval with path-based weighting.
- We describe practical implementation considerations for retrieval-augmented repair, including containerized chunking, text cleaning for scoring, and retrieval fusion mechanisms.

2 Problem Definition

We study automated program repair in the setting defined by SWE-Bench Lite, a curated benchmark of real bug-fixing tasks extracted from open-source Python repositories. Each task is defined by a tuple (R, q, T, P), where R is a snapshot of a real software repository at a given base commit, q is a natural-language problem description describing a bug or missing functionality, T is the repository's test suite, and P is a *fail-to-pass* test patch that exposes the bug by introducing one or more failing tests.

Our experimental pipeline is based on `mini-SWE-agent` (Yang et al., 2024), which provides containerized execution environments, an agent loop for patch generation, and execution-based evaluation via the provided test suites. For code generation, we use the hosted Qwen model `qwen3-coder-480b-a35b-instruct` (Qwen3-Coder-480B-A35B-Instruct) as a fixed LLM backend across all experiments. Keeping the LLM fixed ensures that differences in outcomes can be attributed primarily to the retrieval strategy rather than changes in generation capacity.

Given a task instance (R, q, T, P), the fail-to-pass patch P is first applied to the base repository state R, yielding a version of the repository in which one or more tests in T fail. The goal of automated repair is to generate a patch ΔR such that, when applied on top of this state, all *fail-to-pass* tests pass while all previously passing tests (*pass-to-pass*) continue to pass. This execution-based criterion defines correctness in SWE-Bench.

Importantly, the model does not have direct access to the full contents of R and must rely on retrieved context or exploratory actions within the execution environment. We decompose the repair process into two components: (1) retrieving a small set of code snippets from R that are relevant to q and the observed failure, and (2) generating a correct patch conditioned on q and the retrieved context. In this work, we fix the generation component and focus exclusively on improving the retrieval step.

3 Baseline: No Retrieval

To quantify the contribution of retrieval, we first establish a no-retrieval baseline in which no repository code is injected into the model prompt. The model is provided only with the natural-language problem description and must locate relevant code through indirect interaction with the repository rather than via retrieved context.

The experimental pipeline uses `mini-SWE-agent`, which executes the model inside a containerized environment with the repository mounted locally. Although no retrieval is performed, the agent can issue shell commands (e.g., `grep`, `sed`, `ls`, `cat`) to inspect files and explore the repository structure. This simulates a scenario in which the model must discover relevant code locations through trial-and-error rather than being guided by retrieved context.

This baseline serves three purposes. First, it establishes a lower bound on performance, indicating how much the model can infer from the problem description alone. Second, it provides insight into the limitations of pure LLM reasoning over large, unfamiliar codebases. Third, it allows us to determine whether retrieval is actually necessary for effective repository-scale program repair.

Analysis of baseline behavior shows that the agent often explores irrelevant directories or pro-

poses patches that fail to integrate with existing logic, highlighting the difficulty of repository-scale reasoning without a localization signal. These observations motivate the development of retrieval-based strategies that provide more focused context to the model.

4 Lexical Retrieval with BM25

As a first retrieval-based method, we employ *lexical retrieval* using BM25, a classical information retrieval ranking function that scores documents based on weighted token overlap with a query. BM25 does not model semantic similarity or paraphrasing; instead, it relies on exact or near-exact lexical matches, weighted by term frequency, inverse document frequency, and document length normalization. While this is a known limitation in general natural language retrieval, it aligns well with software bug reports, which frequently contain concrete identifiers such as rule names, function names, configuration keys, file paths, and error messages.

To enable retrieval, the repository is decomposed into a collection of retrievable units. The codebase is recursively traversed and filtered by file extension and directory patterns, after which each file is split into *overlapping chunks of fixed line length* (default: 90 lines with a 20-line overlap). The overlap mitigates boundary effects and preserves local context. Each chunk is treated as an independent document for retrieval. Two representations are maintained per chunk: (i) the raw code snippet injected into the LLM prompt, and (ii) a cleaned representation used exclusively for retrieval scoring. For retrieval scoring, we use a cleaned version of each chunk where top-level docstrings are stripped (enabled by default) and line comments can optionally be stripped; the original unmodified text is kept for prompt injection.

Given a query q and a document (chunk) d, BM25 computes a relevance score as

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{\mathrm{TF}(t,d) \cdot (k_1+1)}{\mathrm{TF}(t,d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)},$$

where $\mathrm{TF}(t,d)$ denotes the term frequency of token t in document d, $|d|$ is the document length, and avgdl is the average document length across the corpus. We use standard hyperparameters $k_1 = 1.5$ and $b = 0.75$. The inverse document frequency term is computed as

$$\mathrm{IDF}(t) = \log\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right),$$

where N is the number of chunks in the corpus and n_t is the number of chunks containing token t.

For each SWE-Bench instance, the *entire natural-language problem statement* is used as the retrieval query. BM25 scores all repository chunks and ranks them by lexical relevance. The top-K ranked chunks (default: $K = 8$) are injected verbatim into the model prompt, providing a focused and localized view of the repository for code generation.

Bug descriptions often reuse the same identifiers that appear in the code, making lexical overlap highly informative for localization. However, BM25 fails when the issue description and the relevant code use divergent vocabulary, as it does not capture semantic equivalence or intent. This limitation motivates the hybrid retrieval approaches described in subsequent sections.

5 Hybrid Lexical–Semantic Retrieval

To address the limitations of purely lexical retrieval, we use a *hybrid lexical–semantic retrieval pipeline* that combines BM25 with embedding-based semantic reranking. The goal of this approach is to improve the quality of injected repository context, particularly in cases where the relevant code does not share exact surface-level tokens with the issue description.

The pipeline follows a standard two-stage retrieval architecture with a subsequent fusion step. First, BM25 is applied over the full set of repository chunks using the SWE-Bench problem statement as the query. All chunks are ranked by lexical overlap, and a top-N candidate pool is selected, with $N = 100$ in our experiments. This stage serves as an efficient filtering mechanism, substantially reducing the search space before applying more computationally expensive semantic models.

In the second stage, dense embeddings are computed for the query and each candidate chunk using the transformer-based sentence embedding model `intfloat/e5-small-v2`. The model is loaded via Hugging Face `AutoModel`. Inputs follow the model's recommended format, with the query prefixed by "`query:`" and code chunks prefixed by "`passage:`". For each input, we extract the final hidden states and apply attention-mask mean pooling so that padded tokens do not contribute to the representation. The resulting vectors are L2-normalized, and semantic relevance is measured using cosine similarity, which reduces to a dot

product between normalized embeddings. This embedding-based reranking enables retrieval of conceptually related code even when lexical overlap with the query is limited.

Lexical and semantic rankings are combined using *Reciprocal Rank Fusion (RRF)*. Given the BM25 rank $\mathrm{rank}_{\mathrm{bm25}}(d)$ and the embedding-based rank $\mathrm{rank}_{\mathrm{emb}}(d)$ of a candidate chunk d, the fused score is computed as

$$\mathrm{RRF}(d) = \frac{1}{k + \mathrm{rank}_{\mathrm{bm25}}(d)} + \frac{1}{k + \mathrm{rank}_{\mathrm{emb}}(d)},$$

where k is a smoothing constant (default: $k = 60$). We use standard, unweighted RRF; the BM25-only baseline does not apply any fusion. After fusion, candidate chunks are ranked by their RRF scores, and the top-K chunks are injected into the model prompt as repository context, with $K = 8$ in our experiments. In this configuration, BM25 provides high-precision candidate selection based on identifier overlap, while embedding-based similarity refines the ordering within the candidate set.

6 Dense Semantic Retrieval

As an alternative to hybrid lexical–semantic retrieval, we implement a *dense semantic retrieval* approach that relies exclusively on embedding-based similarity search. In contrast to the hybrid pipeline, which applies semantic reranking only within a BM25-selected candidate pool, dense retrieval operates directly over a vector index constructed from the entire repository. This design removes dependence on lexical overlap and enables retrieval based purely on semantic similarity between the issue description and code.

6.1 Character-Based Chunking

Dense semantic retrieval uses *character-based chunking* rather than the line-based chunking employed in lexical and hybrid retrieval. Each source file is divided into chunks of fixed character length (default: 2400 characters) with overlap (default: 300 characters). Overlap helps preserve local context when relevant logic spans chunk boundaries. For interpretability and prompt grounding, character offsets are mapped back to approximate line spans, allowing retrieved snippets to be annotated with file paths and line ranges.

6.2 Embedding and Index Construction

Each chunk is embedded using a hosted NVIDIA embedding model (`nv-embed-v1`) accessed via API calls. Query embeddings are generated with `input_type="query"`, while repository chunks are embedded with `input_type="passage"`, following the model's recommended retrieval configuration. All embeddings are L2-normalized to unit length, enabling cosine similarity to be computed via inner product. A FAISS `IndexFlatIP` index is built over the normalized embeddings to support efficient similarity search.

To avoid repeated indexing work across multiple SWE-Bench instances drawn from the same repository snapshot, the dense index is cached at the repository level. The cache key incorporates the repository identifier, base commit hash, and indexing configuration parameters such as chunk size, overlap, and file inclusion rules, allowing the same index to be reused whenever instances share an identical codebase state.

In addition to retrieved code snippets, we prepend a *high-level repository structure summary* to the model prompt. This summary lists top-level directories along with their relative chunk counts and is included before the retrieved snippets. Providing this lightweight structural overview helps the model contextualize retrieved paths and reason about repository organization.

6.3 Path-Based Weighting and Quota Selection

Purely semantic retrieval can overemphasize large test suites or documentation directories, which may contain text semantically similar to bug descriptions but are less likely to contain the fix. To mitigate this effect, we apply a lightweight *path-based weighting* scheme as a structural prior. Chunks located under source directories (e.g., `src/`, `lib/`, or repository-specific source roots such as `sqlfluff/`) receive a positive weight multiplier (default: 1.15), while chunks under `test/`, `tests/`, `docs/`, or `fixtures/` are downweighted (default: 0.85). Retrieved candidates are then ranked by weighted similarity, defined as cosine similarity multiplied by the path weight.

After retrieving an initial candidate pool (default: $\max(10 \times \mathrm{top\_k}, 50)$), we apply a *quota-based selection* strategy to prevent dominance by any single top-level directory. Candidates are grouped by their top-level path component (e.g., `src`, `tests`), and per-bucket quotas constrain the number of selected chunks (defaults: $\lfloor \mathrm{top\_k}/2 \rfloor$ for `src` and $\lfloor \mathrm{top\_k}/4 \rfloor$ for `tests`). The final top-K chunks (with $K = 8$ in our experiments) are selected un-

der these constraints and injected into the model prompt as dense retrieval context.

7 Query Expansion

Query expansion was used to evaluate whether lexical retrieval could be improved by enriching the original problem statement with additional search queries. A Qwen-based instruction-tuned model generated *three* short, code-oriented queries from the issue description, targeting identifiers, file names, and implementation-related terms. Each query was executed independently during retrieval, and the resulting rankings were merged to produce the final set of retrieved code snippets.

In practice, the retrieved contexts closely matched those produced by BM25 using the original problem statement alone. The generated queries largely reused the same lexical anchors already present in the issue text, such as repeated identifiers and configuration terms, resulting in minimal variation in the retrieved files and code regions.

This behavior indicates that query expansion based only on the problem statement does not introduce sufficient new signal to improve retrieval quality beyond standard lexical matching. Without access to information describing how the failure manifests at runtime, retrieval remains constrained by surface-level vocabulary. This limitation motivates the use of stronger signals, such as execution traces and runtime errors, which directly reflect the code paths involved in the failure and enable more precise retrieval.

8 Trace-to-Retrieve: Execution-Guided Context Selection

The key motivation behind this approach is that test failures provide execution-level signals that are directly tied to the source of the bug. A failing test typically produces stack traces that identify specific files and line numbers, exception types and messages that characterize the failure mode, and assertion outputs that highlight discrepancies between expected and observed behavior. Together, these signals provide a more precise basis for localization than problem descriptions alone, as they reflect the actual execution path taken by the program.

We hypothesize that incorporating these execution-level signals into retrieval may lead to more accurate and relevant context selection for downstream code repair. Unlike natural-language issue descriptions, which are often incomplete or loosely specified, runtime traces expose concrete evidence of how the system behaves under failure. As a result, retrieval conditioned on execution information may better align the provided context with the true cause of the bug.

Using execution traces as a retrieval signal has several potential advantages. First, stack-frame information can restrict retrieval to the specific files and code regions involved in the failure, substantially reducing the search space. Second, runtime identifiers such as function names, file paths, and exception types enable the construction of more focused lexical queries than those derived from the problem statement alone. Third, assertion outputs capture semantic mismatches between expected and actual behavior, providing additional context that is typically unavailable in issue text. Collectively, these signals may provide a stronger foundation for context-aware code repair.

Implementation Details. We implement Trace-to-Retrieve within the mini-SWE-agent framework, extending the standard SWE-Bench execution pipeline with an explicit execution and trace-capture stage. For each instance, the official SWE-Bench evaluation container is initialized using the provided Docker image, and the corresponding *fail-to-pass* test patch is applied to the repository. The failing test is then executed in isolation using `pytest`, ensuring that the observed failure corresponds exactly to the benchmark specification.

During execution, we capture the full test output, including standard output, standard error, and the complete Python traceback. From this output, we extract structured runtime signals such as the failing test identifier, stack trace frames, referenced file paths and line numbers, exception types, and assertion messages. These elements are parsed directly from the test output rather than inferred, ensuring that retrieval is grounded in observed execution behavior.

The extracted signals are then used to guide repository retrieval. In particular, file paths and function names appearing in stack frames are treated as high-priority retrieval anchors, while exception messages and assertion text are incorporated as auxiliary lexical cues. Retrieval is performed over the repository contents, with a strong preference for source files referenced in the trace and optional down-weighting of test files to avoid overfitting to the failure specification itself. The

resulting code snippets are injected into the agent prompt alongside the original problem description.

Crucially, the agent loop, model invocation, and patch generation process remain unchanged. The Trace-to-Retrieve method modifies only the retrieval stage, allowing its impact to be evaluated independently of other system components.

Current Status. At the time of writing, Trace-to-Retrieve is under active development and experimental evaluation. The execution and trace-extraction pipeline is fully functional, and failing tests can be reliably reproduced inside the SWE-Bench evaluation environment. Preliminary experiments confirm that execution traces consistently identify the files and code regions involved in failures, even in cases where the natural-language problem description is underspecified.

Current experiments focus on validating the correctness of trace extraction and assessing the qualitative relevance of trace-guided retrieval compared to baseline lexical and semantic retrieval strategies. We do not yet report aggregate resolution metrics. Instead, we prioritize per-instance analysis to ensure pipeline stability and to understand how execution-derived signals influence the retrieved context. Quantitative evaluation across a broader set of instances is planned as future work once the implementation is finalized.

9 Implementation Details

All retrieval operations are performed inside containerized environments provided by SWE-Bench. The repository is mounted at `/testbed` inside the container. Chunking scripts are executed via `env.execute()` calls, ensuring that file system operations occur in the same environment where patches will be applied.

9.1 Why `mini-SWE-agent` under limited resources.

`mini-SWE-agent` is practical in constrained settings because it keeps the agent loop lightweight and relies on standard shell-based interaction within a containerized environment. This reduces engineering overhead while preserving reproducibility through execution-based testing, making it suitable for controlled retrieval experiments without requiring substantial local infrastructure beyond access to the LLM endpoint.

9.2 Text Cleaning for Scoring

To improve retrieval quality, we optionally strip docstrings and comments from the scoring text while preserving them in the injected text. Top-level module docstrings (triple-quoted strings at file start) can be removed from scoring. Line comments (both full-line and inline) can also be stripped. This allows BM25 and embedding models to focus on code structure rather than documentation.

9.3 Caching and Efficiency

Embeddings are cached both in-memory (per Python process) and on-disk (for semantic retrieval). The hybrid approach caches embedder models per (model_name, device) tuple to avoid redundant model loading. Semantic retrieval uses content-addressed chunk IDs to reuse embeddings across commits within the same repository.

10 Experimental Results

We evaluated the following retrieval strategies—a no-retrieval baseline, BM25 lexical retrieval, BM25 with semantic reranking, and dense semantic retrieval—on a subset of 23 instances from the *development split* of the SWE-Bench Lite benchmark. Table 1 summarizes the results for each method. The no-retrieval baseline completed 21 instances and successfully resolved 6. BM25 lexical retrieval completed 20 instances and resolved 6, but produced 3 empty patches. The hybrid BM25+semantic reranking approach completed 20 instances and resolved 5, while the dense semantic retrieval method completed 22 instances and resolved 6, with only a single empty patch.

Table 1: Performance comparison of retrieval strategies.

Method	Sub.	Comp.	**Res.**	Unres.	Empty	Err.	Reg.
Baseline	23	21	**6**	15	0	1	0
BM25	23	20	**6**	14	3	0	0
BM25+Sem	23	20	**5**	15	2	1	0
Semantic	23	22	**6**	16	1	0	0

Overall, the aggregate resolution counts are similar across retrieval strategies and the baseline. However, closer inspection reveals that the set of resolved instances differs between methods. Retrieval-augmented approaches do not simply reproduce the successes of the baseline; instead, they shift which problems are solved. This suggests that retrieval provides complementary strengths rather than uniform improvements.

In particular, retrieval can be beneficial for instances where the problem description contains concrete identifiers or file references that align well with repository structure, enabling faster localization. At the same time, retrieval can bias the model toward plausible but incorrect regions of the codebase, potentially being less effective for tasks that require broader repository-level reasoning or non-local changes.

Among the evaluated methods, semantic retrieval achieved the highest completion rate (22/23) and the lowest empty patch rate, indicating greater robustness in maintaining productive model behavior. However, none of the retrieval strategies demonstrate a clear improvement in resolution rate over the no-retrieval baseline. These results suggest that retrieval alone does not consistently improve patch correctness, and that stronger signals—such as execution-guided retrieval—may be required to achieve consistent gains. We additionally analyze test regressions to assess the safety of retrieval-augmented code generation. A regression is defined as a PASS-to-FAIL transition, where a test that originally passed fails after applying the generated patch. Across all evaluated retrieval strategies—including BM25 lexical retrieval, hybrid lexical–semantic reranking, and dense semantic retrieval—we observe zero regressions on the MiniSWE development set. That is, none of the generated patches caused previously passing tests to fail. This result indicates that while retrieval quality significantly impacts issue resolution rates, incorrect retrieval does not tend to produce destructive patches. Instead, failures are conservative: patches either fail to resolve the issue or fail newly introduced tests, without breaking existing functionality. This suggests that retrieval errors in MiniSWE primarily affect effectiveness rather than safety.

11 Code Availability

To support reproducibility, we release our implementation and experiment scripts at https://github.com/kristinhambardzumyan/mini-swe-agent/tree/rag. The repository includes retrieval modules for BM25, hybrid fusion, dense semantic retrieval, and a preliminary trace-to-retrieve implementation, used in our SWE-Bench Lite experiments.

12 Future Work

Future work will focus on completing a full evaluation of the trace-to-retrieve method on SWE-Bench Lite and understanding when execution-level signals provide the greatest benefit. In particular, we plan to integrate trace-guided signals with both lexical retrieval and dense embedding-based retrieval, and to analyze when such signals are beneficial for retrieval quality versus when they introduce noise. An additional direction worth exploring is multi-stage dense retrieval, where an initial dense retriever produces a larger candidate set that is subsequently refined by a stronger reranking model before context injection.

13 Conclusion

Our results indicate that retrieval based solely on natural-language issue descriptions is not consistently sufficient for reliable code localization on the SWE-Bench Lite development split instances. Although retrieval-augmented methods achieve similar overall resolution rates to the no-retrieval baseline, they succeed on different subsets of instances, suggesting that the issue description alone often provides a weak or mismatched signal for identifying the correct modification location within the codebase.

A promising direction is to incorporate execution-grounded signals obtained from failing tests. Stack traces, exception messages, and assertion outputs expose concrete file paths, function names, and failure locations, offering a more precise localization signal than problem text alone. Building on this insight, future work will explore trace-driven retrieval that executes failing tests prior to patch generation and uses the resulting execution traces to guide retrieval toward failure-adjacent code, with the goal of improving context selection for repository-scale program repair.

References

Carlos E. Jiménez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, Karthik Narasimhan, and Graham Neubig. 2024. Swe-bench: Can language models resolve real-world github issues? In *Proceedings of EMNLP*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and Sebastian Riedel. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *NeurIPS*.

John Yang, Carlos E. Jiménez, Alexander Wettig, Ofir Press, and 1 others. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*.